

# JavaScript if...else Statement

The JavaScript if...else statement is used to execute/skip a block of code based on a condition.

## JavaScript if Statement

We use the if keyword to execute code based on some specific condition.

The syntax of if statement is:

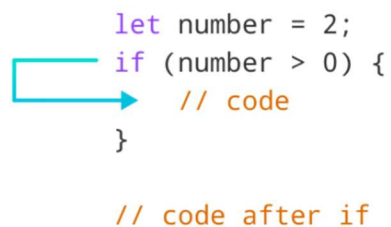
```
if (condition) {  
    // block of code  
}
```

The if keyword checks the condition inside the parentheses ().

- If the condition is evaluated to true, the code inside { } is executed.
- If the condition is evaluated to false, the code inside { } is skipped.

**Note:** The code inside { } is also called the body of the if statement.

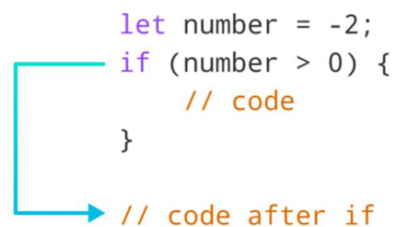
### Condition is true



```
let number = 2;  
if (number > 0) {  
    // code  
}  
  
// code after if
```

The diagram shows a teal arrow starting from the 'if' statement, pointing to the code block inside the curly braces, and then continuing down to the code after the if statement, indicating that the code inside the braces is executed when the condition is true.

### Condition is false



```
let number = -2;  
if (number > 0) {  
    // code  
}  
  
// code after if
```

The diagram shows a teal arrow starting from the 'if' statement, pointing to the code block inside the curly braces, and then continuing down to the code after the if statement, indicating that the code inside the braces is skipped when the condition is false.

## JavaScript else Statement

We use the else keyword to execute code when the condition specified in the preceding if statement evaluates to false.

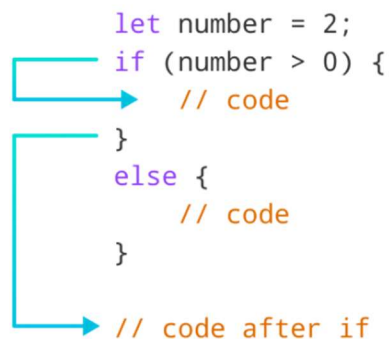
The syntax of the else statement is:

```
if (condition) {  
    // block of code  
    // execute this if condition is true  
}  
else {  
    // block of code  
    // execute this if condition is false  
}
```

The if...else statement checks the condition and executes code in two ways:

- If condition is **true**, the code inside if is executed. And, the code inside else is skipped.
- If condition is **false**, the code inside if is skipped. Instead, the code inside else is executed.

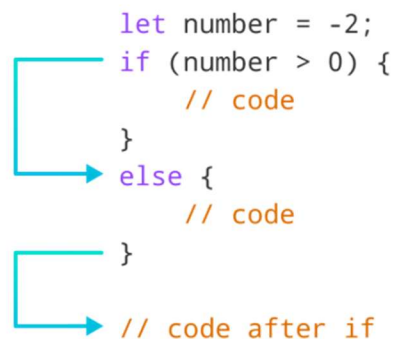
### Condition is true



```
let number = 2;  
if (number > 0) {  
    // code  
}  
else {  
    // code  
}  
// code after if
```

The diagram illustrates the execution flow for the 'Condition is true' scenario. It shows a code snippet with an if-else statement. A blue arrow points from the 'if' condition to the code block inside the if statement. Another blue arrow points from the 'else' block to the code block inside the if statement, indicating that the else block is skipped. A final blue arrow points from the end of the if-else block to the code after the if statement.

### Condition is false



```
let number = -2;  
if (number > 0) {  
    // code  
}  
else {  
    // code  
}  
// code after if
```

The diagram illustrates the execution flow for the 'Condition is false' scenario. It shows a code snippet with an if-else statement. A blue arrow points from the 'if' condition to the code block inside the else statement, indicating that the if block is skipped. Another blue arrow points from the end of the if-else block to the code after the if statement.

## **JavaScript else if Statement**

We can use the else if keyword to check for multiple conditions.

The syntax of the else if statement is:

```
// check for first condition
```

```
if (condition1) {
```

```
    // if body
```

```
}
```

```
// check for second condition
```

```
else if (condition2){
```

```
    // else if body
```

```
}
```

```
// if no condition matches
```

```
else {
```

```
    // else body
```

```
}
```

Here,

1. First, the condition in the if statement is checked. If the condition evaluates to true, the body of if is executed, and the rest is skipped.
2. Otherwise, the condition in the else if statement is checked. If true, its body is executed and the rest is skipped.
3. Finally, if no condition matches, the block of code in else is executed.

### 1st Condition is true

```
let number = 2;
if (number > 0) {
  // code
}
else if (number == 0){
  // code
}
else {
  // code
}
// code after if
```

### 2nd Condition is true

```
let number = 0;
if (number > 0) {
  // code
}
else if (number == 0){
  // code
}
else {
  //code
}
// code after if
```

### All Conditions are false

```
let number = -2;
if (number > 0) {
  // code
}
else if (number == 0){
  // code
}
else {
  //code
}
// code after if
```

## Nested if...else Statement

When we use an if...else statement inside another if...else statement, we create a **nested if...else** statement.

```
let marks = 60;
if (marks >= 40) {
  if (marks >= 80) {
    console.log("Distinction");
  }
  else {
    console.log("Passed");
  }
}
else {
  console.log("Failed");
}
```

// Output: Passed

# JavaScript while and do...while Loop

## JavaScript while Loop

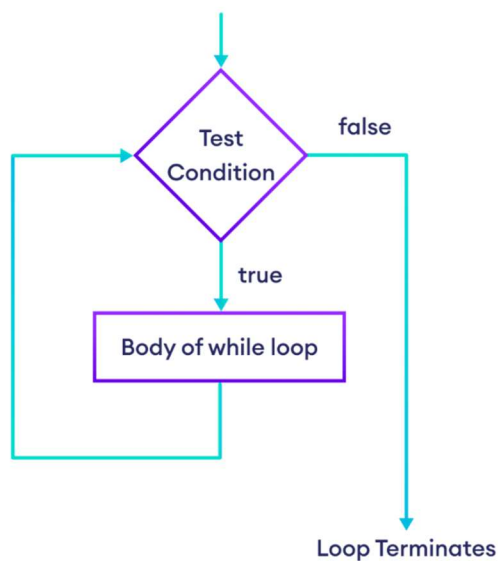
The while loop repeatedly executes a block of code as long as a specified condition is true.

The syntax of the while loop is:

```
while (condition) {  
    // body of loop  
}
```

1. The while loop first evaluates the condition inside ( ).
2. If the condition evaluates to true, the code inside { } is executed.
3. Then, the condition is evaluated again.
4. This process continues as long as the condition evaluates to true.
5. If the condition evaluates to false, the loop stops.

## Flowchart of while Loop



## JavaScript do...while Loop

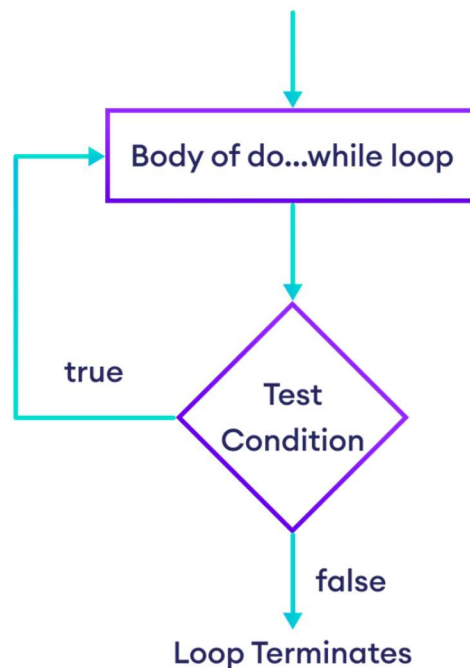
The do...while loop executes a block of code once, then repeatedly executes it as long as the specified condition is true.

The syntax of the do...while loop is:

```
do {  
    // body of loop  
} while(condition);
```

1. The do...while loop executes the code inside { }.
2. Then, it evaluates the condition inside ( ).
3. If the condition evaluates to true, the code inside { } is executed again.
4. This process continues as long as the condition evaluates to true.
5. If the condition evaluates to false, the loop terminates.

## Flowchart of do...while Loop



## JavaScript for loop

In JavaScript, the for loop is used for iterating over a block of code a certain number of times, or to iterate over the elements of an array.

### JavaScript for loop Syntax

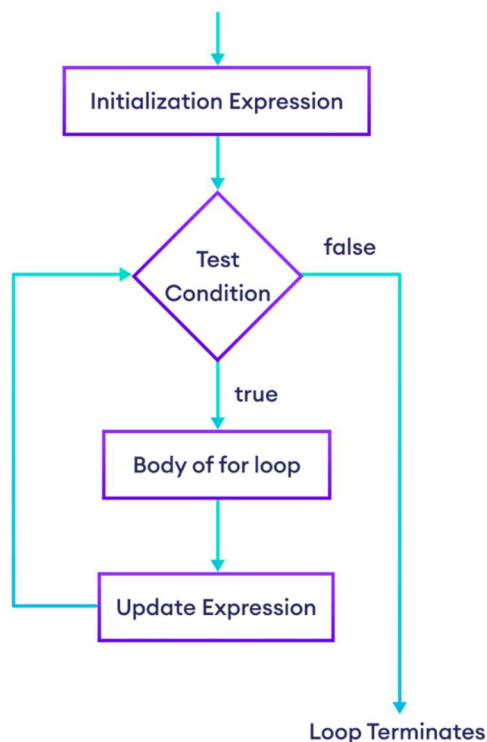
The syntax of the for loop is:

```
for (initialExpression; condition; updateExpression) {  
    // for loop body  
}
```

- initialExpression - Initializes a counter variable.
- condition - The condition to be evaluated. If true, the body of the for loop is executed.
- updateExpression - Updates the value of initialExpression.

Once an iteration of the loop is completed, the condition is evaluated again. The process continues until the condition is false.

### Flowchart of JavaScript for Loop



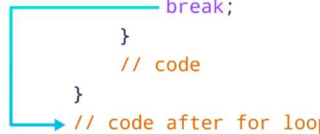
## JavaScript break Statement

The break statement terminates the loop immediately when it's encountered.

### Working of JavaScript break Statement

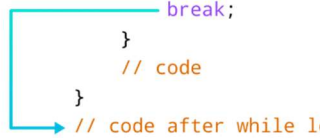
The image below shows the working of the break statement in [for](#) and [while](#) loops.

```
for (initialization; condition; update) {  
    // code  
    if (condition to break) {  
        break;  
    }  
    // code  
}  
// code after for loop
```



---

```
while (condition) {  
    // code  
    if (condition to break) {  
        break;  
    }  
    // code  
}  
// code after while loop
```



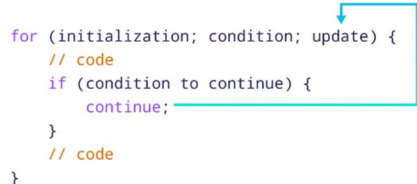
**Note:** The break statement is usually used inside decision-making statements such as [if...else](#).

## JavaScript continue Statement

The continue statement skips the current iteration of the loop and proceeds to the next iteration.

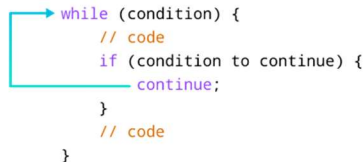
### Working of JavaScript continue Statement

```
for (initialization; condition; update) {  
    // code  
    if (condition to continue) {  
        continue;  
    }  
    // code  
}
```



---

```
while (condition) {  
    // code  
    if (condition to continue) {  
        continue;  
    }  
    // code  
}
```





## **JavaScript for...of loop**

The for..of loop in JavaScript allows you to iterate over iterable objects (arrays, strings etc).

The syntax of the for...of loop is:

```
for (element of iterable) {  
    // body of for...of  
}
```

- **iterable** - an iterable object (array, set, strings, etc).
- **element** - items in the iterable

## **JavaScript for...in loop**

The JavaScript for...in loop iterates over the keys of an object

### **Syntax of JavaScript for...in Loop**

```
for (key in object) {  
    // body of for...in  
};
```

- **object** - The object whose keys we want to iterate over.
- **key** - A variable that stores a single key belonging to object.